Thread_Counter_Report

Lab: Comparing Static and Non-Static Variables in Multithreading (SEPM-001)
Name: Md Jahidul Islam
ID: IT24018
GitHub: https://github.com/mohammadjahidulislam
Directory: Lab_Thread_Counter

Test environment for the results in this report: OpenJDK 21, Linux, 1 CPU core. Command used: `java ThreadCounterLab T N true|false`. Results were taken directly from the program output (see Appendix).

1. Source Code

```
import java.util.concurrent.atomic.AtomicLong;

public class ThreadCounterLab {

    // Shared thread-safe and unsafe static counters
    static AtomicLong safeCounter = new AtomicLong(0);
    static long unsafeCounter = 0;

    // Thread class with a separate local counter
    static class CounterTask extends Thread {
        private final int increments;
        private final boolean threadSafe;
        private long localCounter = 0;

        // Constructor to initialize thread settings
        CounterTask(int increments, boolean threadSafe) {
            this.increments = increments;
            this.threadSafe = threadSafe;
        }

        // Runs when the thread starts
        public void run() {
            for (int i = 0; i < increments; i++) {

                // Increment the static counter
                if (threadSafe) {
                    safeCounter.incrementAndGet();
                } else {
                    unsafeCounter++;
                }

                // Increment the object's local counter
                localCounter++;
            }
        }

        // Return the local counter value
        public long getLocalCounter() {
            return localCounter;
        }
    }

    public static void main(String[] args)
            throws InterruptedException {

        // Check command-line arguments
        if (args.length != 3) {
            System.out.println(
                "Usage: java ThreadCounterLab <threads> <increments> <true|false>"
            );
            return;
        }

        // Read input values
        int n = Integer.parseInt(args[0]);
        int increments = Integer.parseInt(args[1]);
        boolean threadSafe = Boolean.parseBoolean(args[2]);

        // Validate inputs
        if (n <= 0 || increments < 0 ||
                !(args[2].equals("true") ||
                  args[2].equals("false"))) {
            System.out.println("Invalid input.");
            return;
        }
```

```
        // Reset counters before the experiment
        safeCounter.set(0);
        unsafeCounter = 0;

        // Create and start threads
        CounterTask[] tasks = new CounterTask[n];

        for (int i = 0; i < n; i++) {
            tasks[i] = new CounterTask(increments, threadSafe);
            tasks[i].start();
        }

        // Wait for threads and sum local counters
        long nonStaticTotal = 0;

        for (CounterTask task : tasks) {
            task.join();
            nonStaticTotal += task.getLocalCounter();
        }

        // Get the final static counter value
        long staticTotal = threadSafe
                ? safeCounter.get()
                : unsafeCounter;

        // Calculate expected count and difference
        long expected = (long) n * increments;
        long difference = Math.abs(staticTotal - nonStaticTotal);

        // Calculate percentage difference
        double percentage = nonStaticTotal == 0
                ? (difference == 0 ? 0.0 : Double.NaN)
                : difference * 100.0 / nonStaticTotal;

        // Display experiment results
        System.out.println("Threads: " + n);
        System.out.println("Increments per thread: " + increments);
        System.out.println("Mode: " +
                (threadSafe ? "Thread-safe" : "Unsafe"));
        System.out.println("Expected count: " + expected);
        System.out.println("Static count: " + staticTotal);
        System.out.println("Non-static total: " + nonStaticTotal);
        System.out.println("Absolute difference: " + difference);

        // Display percentage difference
        if (Double.isNaN(percentage)) {
            System.out.println("Percentage difference: Undefined");
        } else {
            System.out.printf(
                "Percentage difference: %.4f%%%n", percentage
            );
        }
    }
}
```

2. Test Inputs

Each case was run once in thread-safe mode (`true`) and five times in unsynchronized mode (`false`).

Test	Threads (T)	Increments/thread (N)	Expected = T x N
TC1	1	1,000	1,000
TC2	2	10,000	20,000
TC3	5	10,000	50,000
TC4	10	50,000	500,000
TC5	20	50,000	1,000,000
TC6	50	50,000	2,500,000
TC7	100	50,000	5,000,000

3. Result Analysis

3.1 Experiment A: thread-safe (AtomicLong)

Test	Threads	Expected	Static count	Non-static total	Abs. diff	Diff (%)
TC1	1	1,000	1,000	1,000	0	0.0000
TC2	2	20,000	20,000	20,000	0	0.0000
TC3	5	50,000	50,000	50,000	0	0.0000
TC4	10	500,000	500,000	500,000	0	0.0000
TC5	20	1,000,000	1,000,000	1,000,000	0	0.0000
TC6	50	2,500,000	2,500,000	2,500,000	0	0.0000
TC7	100	5,000,000	5,000,000	5,000,000	0	0.0000

3.2 Experiment B: unsynchronized static counter, five runs per thread count

Threads	Expected	Run	Static count (unsafe)	Non-static total	Abs. diff	Diff (%)
1	1,000	1	1,000	1,000	0	0.0000
		2	1,000	1,000	0	0.0000
		3	1,000	1,000	0	0.0000
		4	1,000	1,000	0	0.0000
		5	1,000	1,000	0	0.0000
2	20,000	1	20,000	20,000	0	0.0000
		2	20,000	20,000	0	0.0000
		3	20,000	20,000	0	0.0000
		4	20,000	20,000	0	0.0000
		5	20,000	20,000	0	0.0000
5	50,000	1	50,000	50,000	0	0.0000
		2	50,000	50,000	0	0.0000
		3	50,000	50,000	0	0.0000
		4	50,000	50,000	0	0.0000

Threads	Expected	Run	Static count (unsafe)	Non-static total	Abs. diff	Diff (%)
		5	50,000	50,000	0	0.0000
10	500,000	1	500,000	500,000	0	0.0000
		2	500,000	500,000	0	0.0000
		3	500,000	500,000	0	0.0000
		4	500,000	500,000	0	0.0000
		5	500,000	500,000	0	0.0000
20	1,000,000	1	1,000,000	1,000,000	0	0.0000
		2	1,000,000	1,000,000	0	0.0000
		3	1,000,000	1,000,000	0	0.0000
		4	1,000,000	1,000,000	0	0.0000
		5	1,000,000	1,000,000	0	0.0000
50	2,500,000	1	2,500,000	2,500,000	0	0.0000
		2	2,500,000	2,500,000	0	0.0000
		3	2,500,000	2,500,000	0	0.0000
		4	2,333,969	2,500,000	166,031	6.6412
		5	2,500,000	2,500,000	0	0.0000
100	5,000,000	1	5,000,000	5,000,000	0	0.0000
		2	5,000,000	5,000,000	0	0.0000
		3	5,000,000	5,000,000	0	0.0000
		4	4,300,000	5,000,000	700,000	14.0000
		5	5,000,000	5,000,000	0	0.0000

3.3 Summary of the five unsynchronized runs

Threads	Expected	Runs with lost updates	Avg. diff (%)	Max diff (%)	Avg. abs. diff
1	1,000	0 / 5	0.0000	0.0000	0
2	20,000	0 / 5	0.0000	0.0000	0
5	50,000	0 / 5	0.0000	0.0000	0
10	500,000	0 / 5	0.0000	0.0000	0
20	1,000,000	0 / 5	0.0000	0.0000	0
50	2,500,000	1 / 5	1.3282	6.6412	33,206
100	5,000,000	1 / 5	2.8000	14.0000	140,000

Difference (%) = |Static - Non-static| / Non-static x 100. Lost updates appeared in 2 of 35 unsynchronized runs.

4. Observations and Conclusion

Thread-safe experiment (control). All seven test cases gave Static count = Non-static total = Expected, with a difference of 0 and 0.0000%. AtomicLong makes every increment indivisible, so no update is lost regardless of thread count.

Unsynchronized experiment. With 1, 2, 5, 10 and 20 threads, all five runs matched the expected count (0%). Lost updates appeared only at higher thread counts: TC6 (50 threads, run 4: 6.6412%) and TC7 (100 threads, run 4: 14.0000%). Averaged over five runs, the difference was 1.3282% for 50 threads and 2.8000% for 100 threads. In these runs the loss appeared only at the largest thread counts, but it hit just 1 run out of 5 in each case, so five runs are too few to claim a steady trend. The difference is not guaranteed to increase with thread count; it depends on scheduling, hardware and run conditions.

Why. `unsafeCounter++` is three steps: read, add one, write back. If a thread is paused between the read and the write, other threads' updates made in the meantime are overwritten when it resumes. Whether this happens depends on when the scheduler interrupts a thread, so it is random. The non-static (local) counters are never shared, so they always total exactly T x N.

Limitation of these runs. They were produced on a machine with one CPU core, where threads are time-sliced rather than running truly in parallel, so races are less frequent than on a multi-core computer. Results on other machines will differ.

Conclusion. Static vs non-static describes ownership (one shared copy vs one copy per object), not thread safety. A static variable is shared, but concurrent updates still need synchronization or atomic operations. Unsynchronized sharing can lose updates unpredictably; AtomicLong removes the problem.

5. Answers to Analysis Questions

Q1. What is the difference between a static and a non-static variable?

A static variable belongs to the class and has one copy shared by all objects. A non-static (instance) variable belongs to an object, and each object has its own copy.

Q2. Why do all threads share the same static counter?

It is stored once with the class, not in any object, so every thread refers to the same memory location.

Q3. Why does each thread have its own non-static counter?

Each thread is a separate CounterTask object, and every object has its own localCounter field.

Q4. Why is join() required before calculating the final counts?

Without join(), main could read the counters while threads are still running and get incomplete values. join() waits until each thread finishes.

Q5. Why can the unsynchronized static count be lower than expected?

unsafeCounter++ is read-modify-write. Two threads can read the same value and both write back value+1, so one increment is lost.

Q6. Does increasing threads always increase the percentage difference?

No. In these runs there was no loss up to 20 threads, and loss appeared only in 1 of 5 runs at 50 and at 100 threads. The result depends on scheduling and hardware, so it is not guaranteed to rise steadily.

Q7. Why might two runs with the same thread count differ?

Thread scheduling and timing change from run to run, so the points where threads interleave and overwrite each other differ.

Q8. What changes when AtomicLong is used?

incrementAndGet() does the read-modify-write as one atomic operation, so no updates are lost and the result is always exact.

Q9. How can all threads share one instance counter too?

Create one shared object (or counter object) and pass it to every thread so they all update the same instance field. That field then also needs AtomicLong or synchronized to be safe.

Appendix: Raw Program Output

The text below is the exact output saved by run_all.sh.

TC1: T=1, N=1,000

```
$ java ThreadCounterLab 1 1000 true
Threads: 1
Increments per thread: 1000
Mode: Thread-safe
Expected count: 1000
Static count: 1000
Non-static total: 1000
Absolute difference: 0
Percentage difference: 0.0000%
```

```
$ java ThreadCounterLab 1 1000 false   (run 1)
Threads: 1
Increments per thread: 1000
Mode: Unsafe
Expected count: 1000
Static count: 1000
Non-static total: 1000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 1 1000 false   (run 2)
Threads: 1
Increments per thread: 1000
Mode: Unsafe
Expected count: 1000
Static count: 1000
Non-static total: 1000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 1 1000 false   (run 3)
Threads: 1
Increments per thread: 1000
Mode: Unsafe
Expected count: 1000
Static count: 1000
Non-static total: 1000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 1 1000 false   (run 4)
Threads: 1
Increments per thread: 1000
Mode: Unsafe
Expected count: 1000
Static count: 1000
Non-static total: 1000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 1 1000 false   (run 5)
Threads: 1
Increments per thread: 1000
Mode: Unsafe
Expected count: 1000
Static count: 1000
Non-static total: 1000
Absolute difference: 0
Percentage difference: 0.0000%
```

TC2: T=2, N=10,000

```
$ java ThreadCounterLab 2 10000 true
Threads: 2
Increments per thread: 10000
Mode: Thread-safe
Expected count: 20000
Static count: 20000
Non-static total: 20000
Absolute difference: 0
Percentage difference: 0.0000%
```

```
$ java ThreadCounterLab 2 10000 false   (run 1)
Threads: 2
Increments per thread: 10000
Mode: Unsafe
Expected count: 20000
Static count: 20000
Non-static total: 20000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 2 10000 false   (run 2)
Threads: 2
Increments per thread: 10000
Mode: Unsafe
Expected count: 20000
```

```
Static count: 20000
Non-static total: 20000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 2 10000 false   (run 3)
Threads: 2
Increments per thread: 10000
Mode: Unsafe
Expected count: 20000
Static count: 20000
Non-static total: 20000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 2 10000 false   (run 4)
Threads: 2
Increments per thread: 10000
Mode: Unsafe
Expected count: 20000
Static count: 20000
Non-static total: 20000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 2 10000 false   (run 5)
Threads: 2
Increments per thread: 10000
Mode: Unsafe
Expected count: 20000
Static count: 20000
Non-static total: 20000
Absolute difference: 0
Percentage difference: 0.0000%
```

TC3: T=5, N=10,000

```
$ java ThreadCounterLab 5 10000 true
Threads: 5
Increments per thread: 10000
Mode: Thread-safe
Expected count: 50000
Static count: 50000
Non-static total: 50000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 5 10000 false   (run 1)
Threads: 5
Increments per thread: 10000
Mode: Unsafe
Expected count: 50000
Static count: 50000
Non-static total: 50000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 5 10000 false   (run 2)
Threads: 5
Increments per thread: 10000
Mode: Unsafe
Expected count: 50000
Static count: 50000
Non-static total: 50000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 5 10000 false   (run 3)
Threads: 5
Increments per thread: 10000
Mode: Unsafe
Expected count: 50000
Static count: 50000
Non-static total: 50000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 5 10000 false   (run 4)
Threads: 5
Increments per thread: 10000
Mode: Unsafe
Expected count: 50000
Static count: 50000
Non-static total: 50000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 5 10000 false   (run 5)
Threads: 5
Increments per thread: 10000
Mode: Unsafe
Expected count: 50000
Static count: 50000
Non-static total: 50000
Absolute difference: 0
Percentage difference: 0.0000%
```

TC4: T=10, N=50,000

```
$ java ThreadCounterLab 10 50000 true
Threads: 10
Increments per thread: 50000
Mode: Thread-safe
Expected count: 500000
Static count: 500000
Non-static total: 500000
Absolute difference: 0
Percentage difference: 0.0000%
```

```
$ java ThreadCounterLab 10 50000 false   (run 1)
Threads: 10
Increments per thread: 50000
Mode: Unsafe
Expected count: 500000
Static count: 500000
Non-static total: 500000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 10 50000 false   (run 2)
Threads: 10
Increments per thread: 50000
Mode: Unsafe
Expected count: 500000
Static count: 500000
Non-static total: 500000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 10 50000 false   (run 3)
Threads: 10
Increments per thread: 50000
Mode: Unsafe
Expected count: 500000
Static count: 500000
Non-static total: 500000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 10 50000 false   (run 4)
Threads: 10
Increments per thread: 50000
Mode: Unsafe
Expected count: 500000
Static count: 500000
Non-static total: 500000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 10 50000 false   (run 5)
Threads: 10
Increments per thread: 50000
Mode: Unsafe
Expected count: 500000
Static count: 500000
Non-static total: 500000
Absolute difference: 0
Percentage difference: 0.0000%
```

TC5: T=20, N=50,000

```
$ java ThreadCounterLab 20 50000 true
Threads: 20
Increments per thread: 50000
Mode: Thread-safe
Expected count: 1000000
Static count: 1000000
Non-static total: 1000000
Absolute difference: 0
Percentage difference: 0.0000%
```

```
$ java ThreadCounterLab 20 50000 false   (run 1)
Threads: 20
Increments per thread: 50000
Mode: Unsafe
Expected count: 1000000
Static count: 1000000
Non-static total: 1000000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 20 50000 false   (run 2)
Threads: 20
Increments per thread: 50000
Mode: Unsafe
Expected count: 1000000
Static count: 1000000
Non-static total: 1000000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 20 50000 false   (run 3)
Threads: 20
```

```
Increments per thread: 50000
Mode: Unsafe
Expected count: 1000000
Static count: 1000000
Non-static total: 1000000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 20 50000 false   (run 4)
Threads: 20
Increments per thread: 50000
Mode: Unsafe
Expected count: 1000000
Static count: 1000000
Non-static total: 1000000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 20 50000 false   (run 5)
Threads: 20
Increments per thread: 50000
Mode: Unsafe
Expected count: 1000000
Static count: 1000000
Non-static total: 1000000
Absolute difference: 0
Percentage difference: 0.0000%
```

TC6: T=50, N=50,000

```
$ java ThreadCounterLab 50 50000 true
Threads: 50
Increments per thread: 50000
Mode: Thread-safe
Expected count: 2500000
Static count: 2500000
Non-static total: 2500000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 50 50000 false   (run 1)
Threads: 50
Increments per thread: 50000
Mode: Unsafe
Expected count: 2500000
Static count: 2500000
Non-static total: 2500000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 50 50000 false   (run 2)
Threads: 50
Increments per thread: 50000
Mode: Unsafe
Expected count: 2500000
Static count: 2500000
Non-static total: 2500000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 50 50000 false   (run 3)
Threads: 50
Increments per thread: 50000
Mode: Unsafe
Expected count: 2500000
Static count: 2500000
Non-static total: 2500000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 50 50000 false   (run 4)
Threads: 50
Increments per thread: 50000
Mode: Unsafe
Expected count: 2500000
Static count: 2333969
Non-static total: 2500000
Absolute difference: 166031
Percentage difference: 6.6412%

$ java ThreadCounterLab 50 50000 false   (run 5)
Threads: 50
Increments per thread: 50000
Mode: Unsafe
Expected count: 2500000
Static count: 2500000
Non-static total: 2500000
Absolute difference: 0
Percentage difference: 0.0000%
```

TC7: T=100, N=50,000

```
$ java ThreadCounterLab 100 50000 true
Threads: 100
Increments per thread: 50000
```

```
Mode: Thread-safe
Expected count: 5000000
Static count: 5000000
Non-static total: 5000000
Absolute difference: 0
Percentage difference: 0.0000%
```

```
$ java ThreadCounterLab 100 50000 false   (run 1)
Threads: 100
Increments per thread: 50000
Mode: Unsafe
Expected count: 5000000
Static count: 5000000
Non-static total: 5000000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 100 50000 false   (run 2)
Threads: 100
Increments per thread: 50000
Mode: Unsafe
Expected count: 5000000
Static count: 5000000
Non-static total: 5000000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 100 50000 false   (run 3)
Threads: 100
Increments per thread: 50000
Mode: Unsafe
Expected count: 5000000
Static count: 5000000
Non-static total: 5000000
Absolute difference: 0
Percentage difference: 0.0000%

$ java ThreadCounterLab 100 50000 false   (run 4)
Threads: 100
Increments per thread: 50000
Mode: Unsafe
Expected count: 5000000
Static count: 4300000
Non-static total: 5000000
Absolute difference: 700000
Percentage difference: 14.0000%

$ java ThreadCounterLab 100 50000 false   (run 5)
Threads: 100
Increments per thread: 50000
Mode: Unsafe
Expected count: 5000000
Static count: 5000000
Non-static total: 5000000
Absolute difference: 0
Percentage difference: 0.0000%
```